

Global Engineering Solutions

FEST[®]

FEST Library Reference Manual

1. Hex2Int	page 2
2. Int2Hex.....	page 3
3. IsDecimal.....	page 4
4. IsHex.....	page 5
5. IsReal.....	page 6
6. IsExp.....	page 7
7. IsNumeric.....	page 8
8. StrCmpCase.....	page 9
9. StrCmpNoCase.....	page 10
10. StrNCmpCase.....	page 11
11. StrNCmpNoCase.....	page 12
12. StrCmpCaseR.....	page 13
13. StrCmpNoCaseR.....	page 14
14. StrNCmpCaseR.....	page 15
15. StrNCmpNoCaseR.....	page 16
16. FindSubStr.....	page 17
17. FindSubStrNoCase.....	page 18
18. StrCheck.....	page 19
19. SplitStr.....	page 20
20. Str2Token.....	page 21
21. Str2TokenEx.....	page 22



1. Hex2Int

- **Description**

- 문자로된 hex값을 십진수로 변환하여 주는 함수.

- **Syntax**

- int Hex2Int(char* Hex);

- **Parameter**

- Hex : 십진수로 변환할 Hex 문자열

- **Return**

- int -> 10 진수

- **Sample**

- ```
int iData;
char strTest[64];
sprintf(strTest, "2A");
iData=Hex2Int(strTest);
```

---

## 2.Int2Hex

---

- **Description**

- int형 값을 hex 문자열로 변환

- **Syntax**

char\* Int2Hex( int iData , char\* szHexRtn , int nNumOfHex );

- **Parameter**

- iData: 변환할 int 값
- szHexRtn: 변환된 Hex 문자열
- nNumOfHex: return 할 Hex 문자열의 길이(byte)

- **Return**

- 변환된 hex 문자열

- **Sample**

---

## 3.IsDecimal

---

### ● Description

- char 문자열이 Decimal 값으로 이루어져 있는지 확인하는 함수.

### ● Syntax

- BOOL IsDecimal( char \*szDec );

### ● Parameter

- Nothing

### ● Return

- true : 문자열이 Decimal 값으로 이루어 졌을 경우
- false : true 의 반대 경우

### ● Sample

```
char strTest[256];
sprintf(strTest , "123");
if(IsDecimal(strTest)) return atoi(strTest);
```

---

## 4. IsHex

---

- **Description**

- char 문자열이 Hex 값으로 이루어져 있는지 확인하는 함수.

- **Syntax**

- BOOL IsHex( char \*szHex );

- **Parameter**

- Nothing

- **Return**

- true : 문자열이 Decimal 값으로 이루어 졌을 경우
  - false : true 의 반대 경우

- **Sample**

- char strTest[256];  
sprintf( strTest , "123" );  
IsHex(strTest);

---

## 5.IsReal

---

### ● Description

- char 문자열이 실수 값으로 이루어져 있는지 확인하는 함수.

### ● Syntax

```
BOOL IsReal(char *szReal);
```

### ● Parameter

- Nothing

### ● Return

- true : 문자열이 Decimal 값으로 이루어 졌을 경우
- false : true 의 반대 경우

### ● Sample

```
char strTest[256];
sprintf(strTest , "123.0");
IsReal(strTest);
```

---

## 6. IsExp

---

- **Description**

- char 문자열이 지수값으로 이루어져 있는지 확인하는 함수.

- **Syntax**

- BOOL IsExp( char \*szExp );

- **Parameter**

- Nothing

- **Return**

- true : 문자열이 Decimal 값으로 이루어 졌을 경우
  - false : true 의 반대 경우

- **Sample**

- char strTest[256];  
sprintf( strTest , "123" );  
IsExp(strTest);



---

## 7. IsNumeric

---

### ● Description

- char 문자열이 Numeric값으로 이루어져 있는지 확인하는 함수.

### ● Syntax

```
int IsNumeric(char *szNum); // return value means numeric type
```

### ● Parameter

- Nothing

### ● Return

- true : 문자열이 Decimal 값으로 이루어 졌을 경우
- false : true 의 반대 경우

### ● Sample

```
char strTest[256];
sprintf(strTest , "123");
IsNumeric(strTest);
```

---

## 8. StrCmpCase

---

### ● Description

- 대소문자 구분을 하여 두 문자열을 비교 하는 함수

### ● Syntax

- `BOOL StrCmpCase(char* src , char* des);`

### ● Parameter

- Src : 문자열 Source
- Des : 문자열 Destination

### ● Return

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

### ● Sample

```
BOOL STRCMP_L(char *Source , char *Destination) {
 return StrCmpCase(Source , Destination);
}
```

---

## 9. StrCmpNoCase

---

### ● Description

- 대소문자 구분 없이 두 문자열을 비교 하는 함수

### ● Syntax

- `BOOL StrCmpNoCase(char* src , char*des);`

### ● Parameter

- Src : 문자열 Source
- Des : 문자열 Destination

### ● Return

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

### ● Sample

```
BOOL STRCMP(char *Source , char *Destination) {
 return StrCmpNoCase(Source , Destination);
}
```

---

## 10. StrNCmpCase

---

### ● Description

- 대소문자 구분을 하여 두 문자열을 지정한 글자 수 만큼 비교 하는 함수

### ● Syntax

- `BOOL StrNCmpCase(char* src , char* des , size_t ncmp);`

### ● Parameter

- Src : 문자열 Source
- Des : 문자열 Destination
- ncmp : 비교할 문자 개수

### ● Return

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

### ● Sample

```
BOOL STRNCMP_L(char *Source , char *Destination , int CmpCount) {
 return StrNCmpCase(Source , Destination , CmpCount);
}
```

---

## 11. StrNCmpNoCase

---

### ● Description

- 대소문자 구분 없이 두 문자열을 지정한 글자 수 만큼 비교 하는 함수

### ● Syntax

- `BOOL StrNCmpNoCase(char* src , char*des , size_t ncmp);`

### ● Parameter

- Src : 문자열 Source
- Des : 문자열 Destination
- ncmp : 비교할 문자 개수

### ● Return

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

### ● Sample

```
BOOL STRNCMP(char *Source , char *Destination , int CmpCount) { // Ignore for large & Small Character
 return StrNCmpNoCase(Source , Destination , CmpCount);
}
```

---

## 12. StrCmpCaseR

---

- **Description**

- 두 문자열의 대/소 문자 구분하여 오른쪽 끝에서 부터 같은지 검사 하는 함수

- **Syntax**

```
BOOL StrCmpCaseR(char* src,char* desR);
```

- **Parameter**

- Src : 문자열 Source
- DesR : 문자열 Destination

- **Return**

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

- **Sample**

```
BOOL STRCMP(char *Source , char *Destination) {
 return StrCmpCaseR(Source , Destination);
}
```

---

## 13. StrCmpNoCaseR

---

- **Description**

- 두 문자열의 대/소 문자 구분 없이 오른쪽 끝에서부터 같은지 검사 하는 함수

- **Syntax**

```
BOOL StrCmpNoCaseR(char* src,char* desR);
```

- **Parameter**

- Src : 문자열 Source
- DesR : 문자열 Destination

- **Return**

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

- **Sample**

```
BOOL STRCMP(char *Source , char *Destination) {
 return StrCmpNoCaseR(Source , Destination);
}
```

---

## 14. StrNCmpCaseR

---

- **Description**

- 두 문자열의 대/소 문자 구분하여 오른쪽 끝에서부터 개수만큼 같은지 검사 하는 함수

- **Syntax**

```
BOOL StrNCmpCaseR(char* src,char* desR,size_t ncmp);
```

- **Parameter**

- Src : 문자열 Source
- DesR : 문자열 Destination
- ncmp : compare 문자열 개수

- **Return**

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

- **Sample**

```
BOOL STRCMP(char *Source , char *Destination, int ncmp) {
 return StrNCmpCaseR (Source , Destination, ncmp);
}
```



---

## 15. StrNCmpNoCaseR

---

- **Description**

- 두 문자열의 대/소 문자 구분없이 오른쪽 끝에서부터 갯수만큼 같은지 검사 하는 함수

- **Syntax**

```
BOOL StrNCmpNoCaseR(char* src,char* desR,size_t ncmp);
```

- **Parameter**

- Src : 문자열 Source
- DesR : 문자열 Destination
- ncmp : compare 문자열 개수

- **Return**

- false : 두 문자열이 일치할 경우
- true : 두 문자열이 일치하지 않을 경우

- **Sample**

```
BOOL STRCMP(char *Source , char *Destination, int ncmp) {
 return StrNCmpNoCaseR(Source , Destination, ncmp);
}
```

---

## 16. FindSubStr

---

- **Description**

- 대소문자 구분하여 문자열에서 찾고자 하는 문자열 이후부터 찾아주는 함수.

- **Syntax**

- char\* FindSubStr(char\* szSrc, char\* szTarget);

- **Parameter**

- szSrc : 문자열 Source
- szTarget : 찾고자 하는 문자열

- **Return**

- 찾고자 하는 문자열 이후의 문자열

- **Sample**

FindSubStr(\_TEXT("test"), \_TEXT("e"))

---

## 17. FindSubStrNoCase

---

- **Description**

- 대소문자 구분 없이 문자열에서 찾고자 하는 문자열 이후부터 찾아주는 함수.

- **Syntax**

char\* FindSubStrNoCase(char\* szSrc, char\* szTarget);

- **Parameter**

- szSrc : 문자열 Source
- szTarget : 찾고자 하는 문자열

- **Return**

- 찾고자 하는 문자열 이후의 문자열

- **Sample**

FindSubStrNoCase (\_TEXT("test"), \_TEXT("E"))

---

## 18. StrCheck

---

### ● Description

- 문자열에 비정상적인 문자열이 포함 되어 있는지 확인하는 함수.

### ● Syntax

- `BOOL StrCheck( char* szSrc , char* szAbnormalCharSet );`

### ● Parameter

- `szSrc` : 문자열 Source
- `szAbnormalCharSet` : 소스 문자열에 포함되지 않을 비정상적인 문자 집합

### ● Return

- `true` : 비정상적인 문자를 포함하지 않은 경우
- `false` : 비정상적인 문자를 포함한 경우

### ● Sample

`StrCheck(_TEXT("!test") , _TEXT("!"))`

---

## 19. SplitStr

---

### ● Description

- source 문자열을 특정 문자로 구분하여 분리해 내는 함수

### ● Syntax

- char\* SplitStr( char\* Src , char\* szToken , char\* szSplitters , int nTrim );

### ● Parameter

- Src : [IN] source 문자열
- szToken : [OUT] 문자열을 포함하는 토큰
- szSplitters : [IN] 구분 문자 집합
- nTrim : [IN] 공백 트리밍 옵션 선택 0:don't trim , 1:left trim , 2:right trim , 3:trim all

### ● Return

- 첫번째 구분 기호에 대한 포인터
- 구분 기호를 찾을 수 있지만, SRC 는 유효한 문자열이 없는 경우 \*Return = NULL
- 구분 기호를 찾을 수 없는 경우, SRC 는 유효한 문자열이 없는 경우 \*Return = NULL

### ● Sample

SplitStr( strTest , \_TEXT("|") , \_TEXT(" ") , 3 )

---

## 20. Str2Token

---

### ● Description

- source 문자열을 szSplitters 문자열에 포함 된 문자로 분리해 내는 함수.

### ● Syntax

- char\* Str2Token( char\* Src , char\* szToken , char\* szSplitters , char\* szExcludeChars , int nTrim );

### ● Parameter

- Src : [IN] source 문자열
- szToken : [OUT] 문자열을 포함하는 토큰
- szSplitters : [IN] 구분 문자 집합
- szExcludeChars : [IN] 토큰에 포함 할 필요가 없는 문자 집합
- nTrim : [IN] 공백 없애는 옵션 선택(0:don't trim , 1:left trim , 2:right trim , 3:trim all)

### ● Return

- 첫번째 구분 기호에 대한 포인터
- 구분 기호를 찾을 수 있지만, SRC 는 유효한 문자열이 없는 경우 \*Return = NULL
- 구분 기호를 찾을 수 없는 경우, SRC 는 유효한 문자열이 없는 경우 \*Return = NULL
- 만약에 일치한 AtomEnd 문자를 찾을수 없는 경우 \*Return = NULL

### ● Sample

---

## 21. Str2TokenEx

---

### ● Description

- source 문자열을 szSplitters 문자열이 포함 된 문자로 분리해 낸다.
- szAtomS , szAtomE 문자열은 토큰에 포함되지 않는다.

### ● Syntax

- char\* Str2TokenEx( char\* Src , char\* szToken , char\* szSplitters , char\* szExcludeChars , char\* szAtomS , char\* szAtomE , int nTrim );

### ● Parameter

- Src : [IN] source 문자열
- szToken : [OUT] 문자열을 포함하는 토큰
- szSplitters : [IN] 구분 문자 집합
- szExcludeChars : [IN] 토큰에 포함 할 필요가 없는 문자 집합
- nTrim : [IN] 공백 트리밍 옵션 선택 0:don't trim , 1:left trim , 2:right trim , 3:trim all
- szAtomS : [IN] 시작하는 문자를 나타내고, szAtomS , szAtomE 구분은 분석 되지 않는다.
- szAtomE : [IN] 끝 문자를 나타내고, szAtomS , szAtomE 구분은 분석 되지 않는다

### ● Return

- 첫번째 구분 기호에 대한 포인터
- 구분 기호를 찾을 수 있지만, SRC 는 유효한 문자열이 없는 경우 \*Return = NULL
- 구분 기호를 찾을 수 없는 경우, SRC 는 유효한 문자열이 없는 경우 \*Return = NULL
- 만약에 일치한 AtomEnd 문자를 찾을수 없는 경우 \*Return = NULL

### ● Sample

---

URL : <http://www.gesolutions.co.kr/>

(주)게스

우 305-500

대전광역시 유성구 용산동 533-1번지 미건테크노월드Ⅱ C동 440호

전화 : 042-671-3331

G.E.S

#440C-Gate Migun TechnoWorld Ⅱ, 533-1

Yongsan-dong, Yuseong-gu, Daejeon, Korea

Tel : +82-42-671-3331

